

MINIPROJETO: SISTEMA DE MONITORIZAÇÃO

Feito por:

Ricardo António Parracha Pulido, Nº Mec 131393

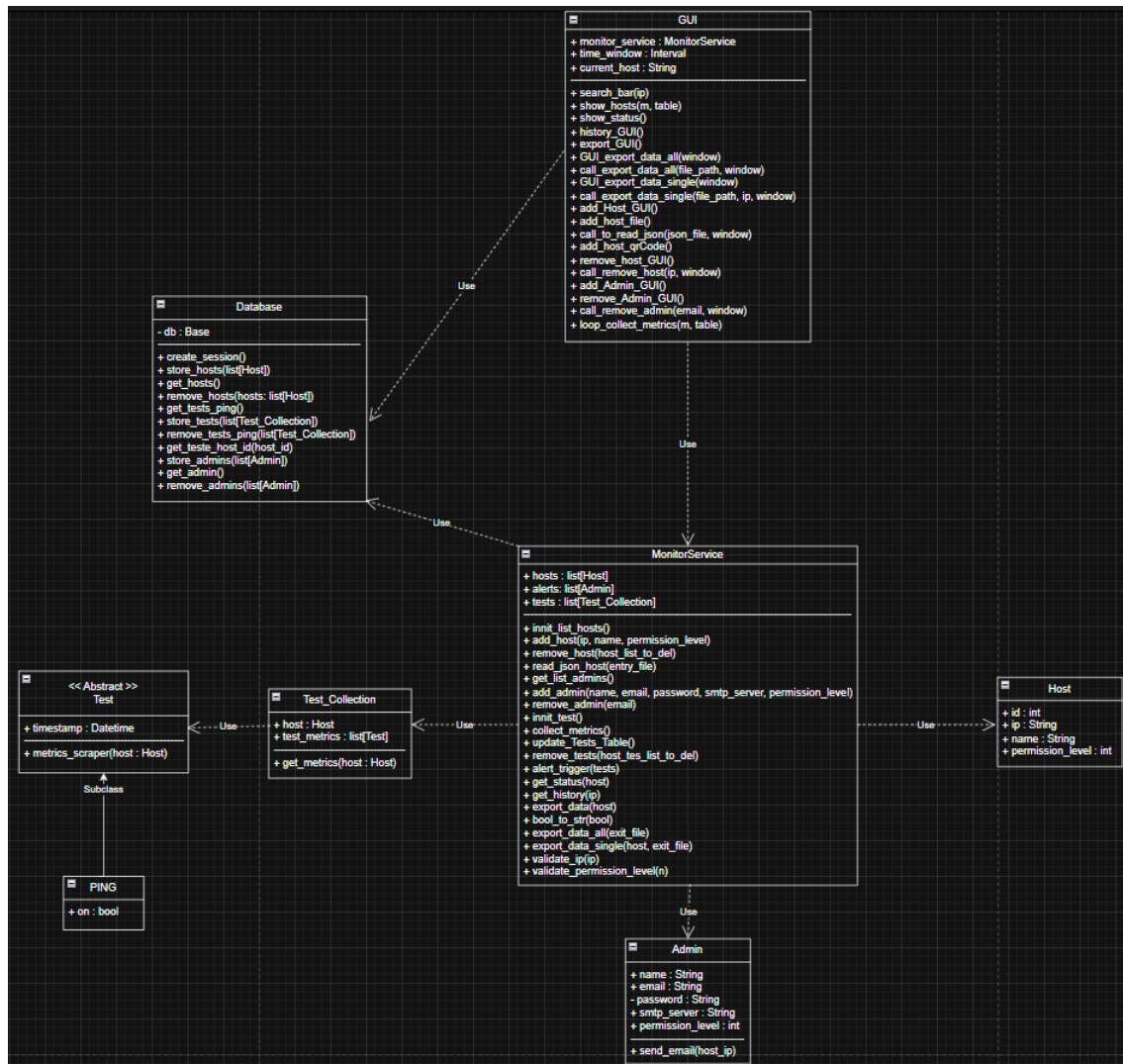
Inês Nunes Santos, Nº Mec 103477

Simão Oliveira Tavares, Nº Mec 131330

Programação
Aplicada, ESTGA
2025/2026

CTeSP em Redes e
Sistemas
Informáticos

Diagrama de Classes

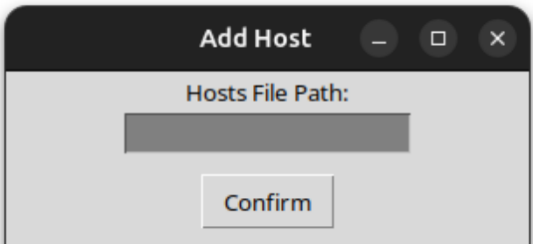
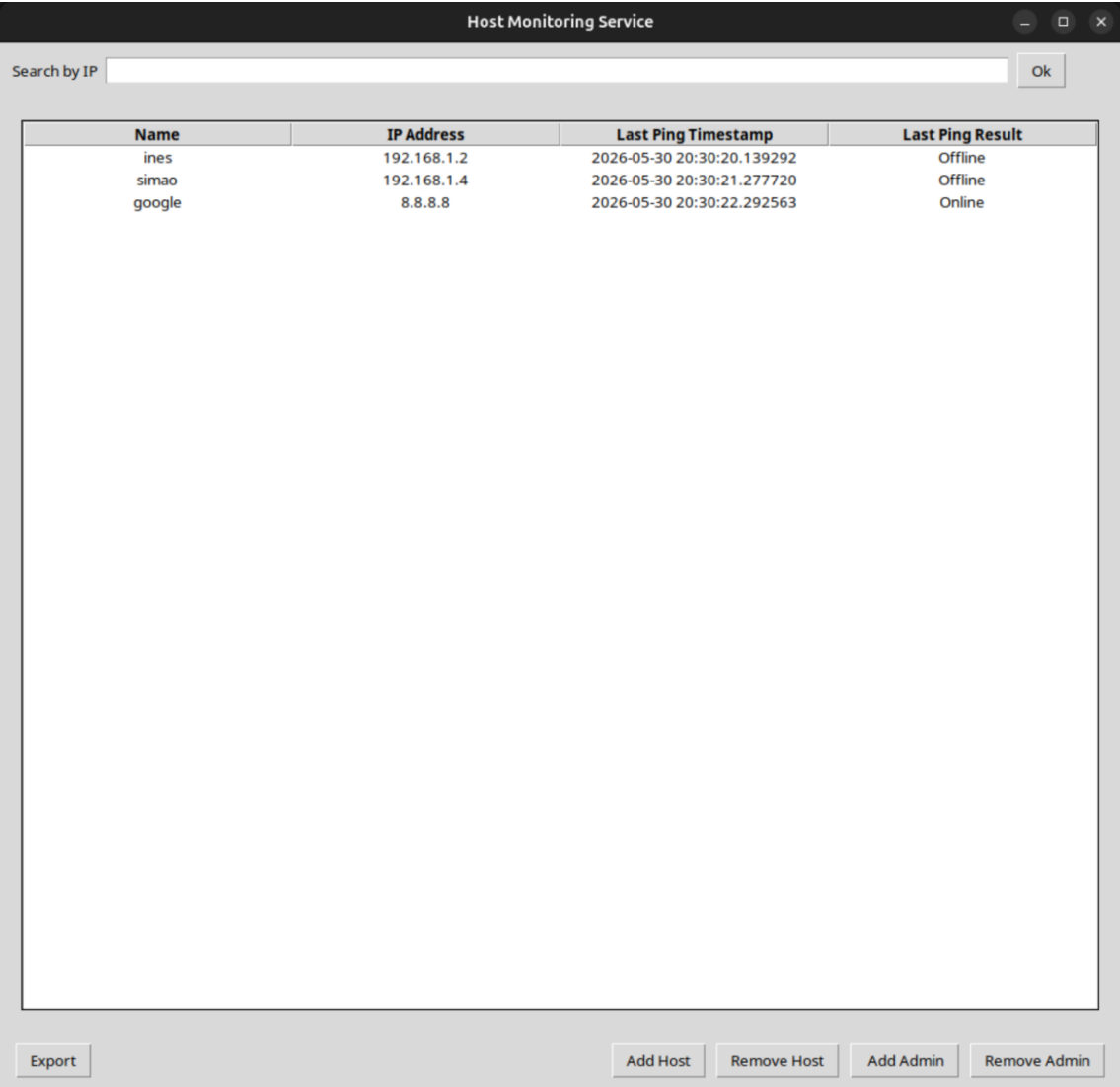


Estratégia para a composição dos componentes

Ao pensar o UML, decidimos dividi-lo em 3 classes principais que se usavam entre si: a Database, a classe MonitorService e a classe GUI.

A classe MonitorService (que usa a classe Database, e é utilizada pela classe GUI), é o centro da aplicação de monitorização de máquinas, é ela que faz a gestão de que modo os dados vão ser guardados pela Database, a validação de alguns dados, criação e gestão de objetos que vão ser usados pelo GUI.

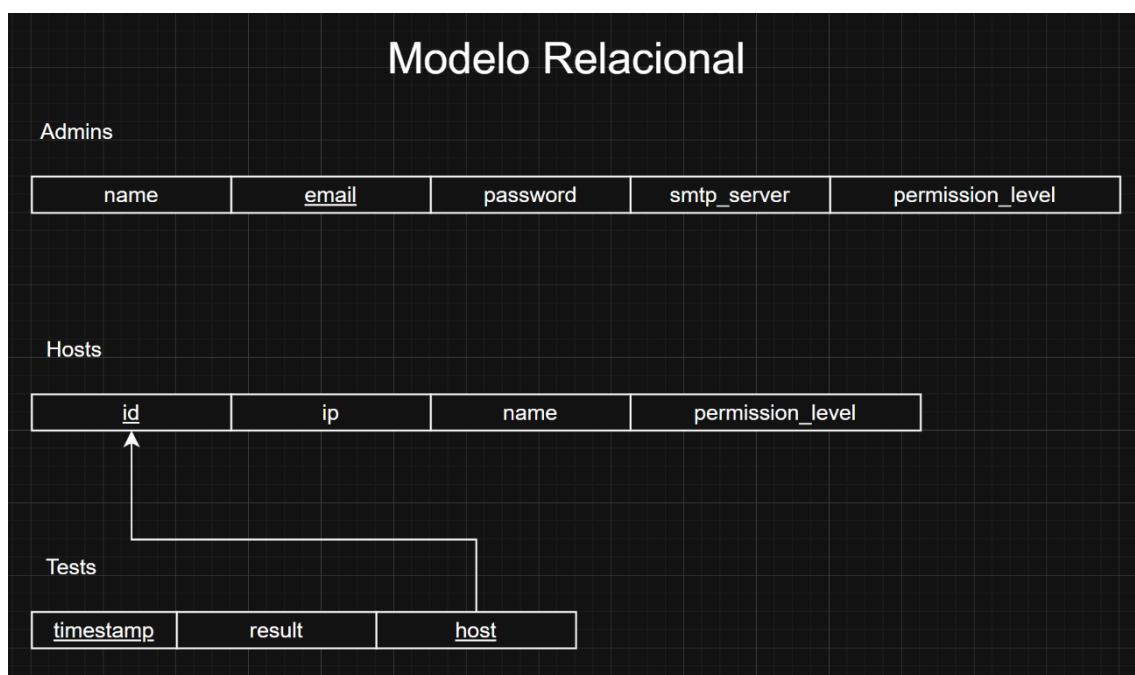
A classe **GUI** (Graphical User Interface), implementada através de Tkinter em Python3, responsável pelo ambiente de visualização e pela interatividade dos dados, faz uso da Database e do MonitorService, chamando as suas funções no momento em que o utilizador pretender realizar determinadas ações de monitorização. Por exemplo: ao clicar no botão “Add Host”, será chamada a função `add_host_file` de modo a que o utilizador possa adicionar uma máquina através de um ficheiro de configuração, que depois será responsável por chamar a função correspondente da base de dados para armazenar essa informação.



Além destas 3 classes, foram criadas as classes Admin e Host, que vão ser utilizadas pelo Monitor Service, para ser possível fazer a correspondência entre objetos e os dados desses mesmos objetos.

Por fim, o último conjunto de classes: a classe abstrata Test, que define um timestamp por cada teste feito, e o seu método metrics_scrapped, que terá que ser implementado pela sua subclasse Ping (que terá um resultado); e a classe Test_Collection, que usa a classe Test para ir buscar e armazenar os resultados (metrics) de cada host numa lista de testes. A classe Test_Collection é depois usada pelo MonitorService para a aplicação conseguir tratar os testes de cada host (visualizando-os no GUI através de um gráfico).

Modelo Relacional das Entidades da Base de Dados



Para o Modelo Relacional das 3 entidades/tabelas (Admins, Hosts e Tests), definimos o campo 'email' como chave primária (PK) de cada Admin, o campo 'id' como chave primária de cada Host, e os campos 'timestamp' e 'host' como chave primária composta de cada Test. Além disto, relacionámos as tabelas 'Hosts' e 'Tests' definindo o host_id como chave estrangeira (FK) da tabela Tests, pois cada host/máquina poderá realizar múltiplos testes/pings (relação de 1 : N).

Descrição da DB

A classe Database, feita em sqlalchemy, seria constituída por 3 tabelas (Hosts, Admins e Tests), e teria múltiplas funções: para criar a sessão da base de dados; guardar informação de máquinas/hosts, admins e testes (Pings, neste caso) de cada host; aceder a hosts, admins e testes; e para remover hosts, admins e testes.

Descrição do processo de monitorização

Ao inicializar o programa de monitorização (executando o ficheiro GUI.py no terminal), vamos à base de dados verificar se ela tem informação guardada, através do construtor Monitor Service. Se tiver, guardamos essa informação em memória.

```
GUI.py > call_to_read_json
451
452
453 if __name__ == "__main__":
454
455     # Search Bar:
456     frame_search_bar = tk.Frame(main_window)
457     frame_search_bar.pack(fill="x", padx=10, pady=8)
458     tk.Label(frame_search_bar, text="Search by IP").pack(side="left")
459     search_entry = tk.Entry(frame_search_bar, width=100)
460     search_entry.pack(side="left", padx=4)
461     tk.Button(frame_search_bar, text="Ok", command=lambda : search_bar(search_entry.get())).pack(side="left", padx=2)
462
463
464
465
466     # zona para as duas frames (gráfico e Lista de hosts):
467     frame_hosts = tk.Frame(main_window, bg="#d9d9d9", highlightbackground="black", highlightthickness=1)
468
469     # Criar Tabela
470     table = ttk.Treeview(frame_hosts)
471
472     table["columns"] = ("Name", "IP Address", "Last Ping Timestamp", "Last Ping Result")
473
474     table.column('#0', width=0, stretch=tk.NO)
475     table.column('Name', anchor=tk.CENTER, width=200)
476     table.column('IP Address', anchor=tk.CENTER, width=200)
477     table.column('Last Ping Timestamp', anchor=tk.CENTER, width=200)
478     table.column('Last Ping Result', anchor=tk.CENTER, width=200)
```

```

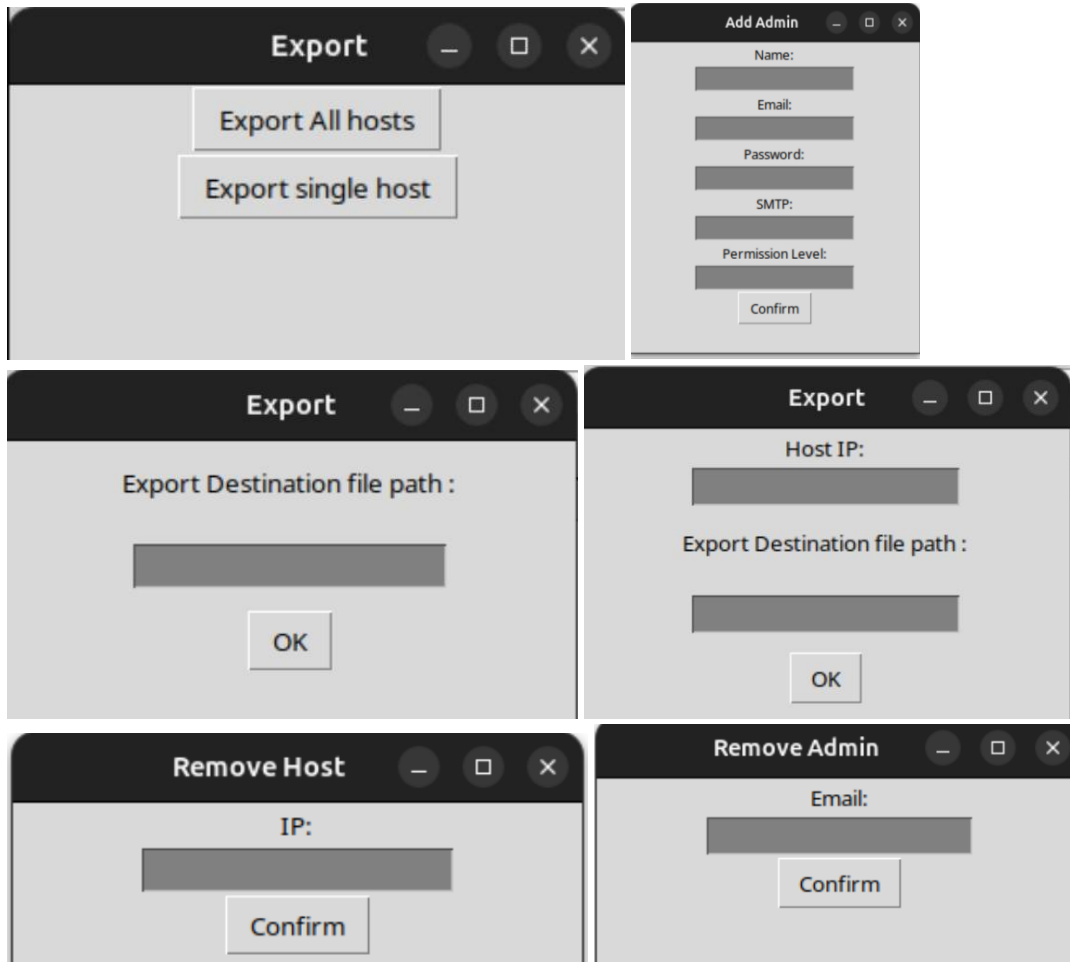
GUI.py > ...
479
480 table.heading('#0', text='', anchor=tk.CENTER)
481 table.heading('Name', text='Name', anchor=tk.CENTER)
482 table.heading('IP Address', text='IP Address', anchor=tk.CENTER)
483 table.heading('Last Ping Timestamp', text='Last Ping Timestamp', anchor=tk.CENTER)
484 table.heading('Last Ping Result', text='Last Ping Result', anchor=tk.CENTER)
485
486 table.tag_configure('oddrow', background=■ '#E8E8E8')
487 table.tag_configure('evenrow', background=■ '#FFFFFF')
488
489
490 frame_hosts.pack(fill="both", expand=True, padx=20, pady=20)
491
492
493 # Barra de botões inferiores:
494 frame_Buttons = tk.Frame(main_window)
495 frame_Buttons.pack(side="bottom", fill="x", padx=10, pady=8)
496
497
498 tk.Button(frame_Buttons, text="Export", command=export_GUI).pack(side="left", padx=4)
499 tk.Button(frame_Buttons, text="Remove Admin", command=remove_Admin_GUI).pack(side="right", padx=4)
500 tk.Button(frame_Buttons, text=" Add Admin", command=add_Admin_GUI).pack(side="right", padx=4)
501 tk.Button(frame_Buttons, text="Remove Host", command=remove_host_GUI).pack(side="right", padx=4)
502 tk.Button(frame_Buttons, text="Add Host", command=add_Host_GUI).pack(side="right", padx=4)
503
504
505 m = monitor()
506
507 loop_collect_metrics(m,table)
508
509 monitor.innit_list_hosts(m)
510
511
512 main_window.mainloop()
513
514 Database.engine.dispose()

```

Periodicamente, de 4 em 4 segundos, temos a função 'loop_collect_metrics' a realizar testes (pings) para cada host/máquina adicionado ao programa, guardando assim os respectivos resultados na tabela.

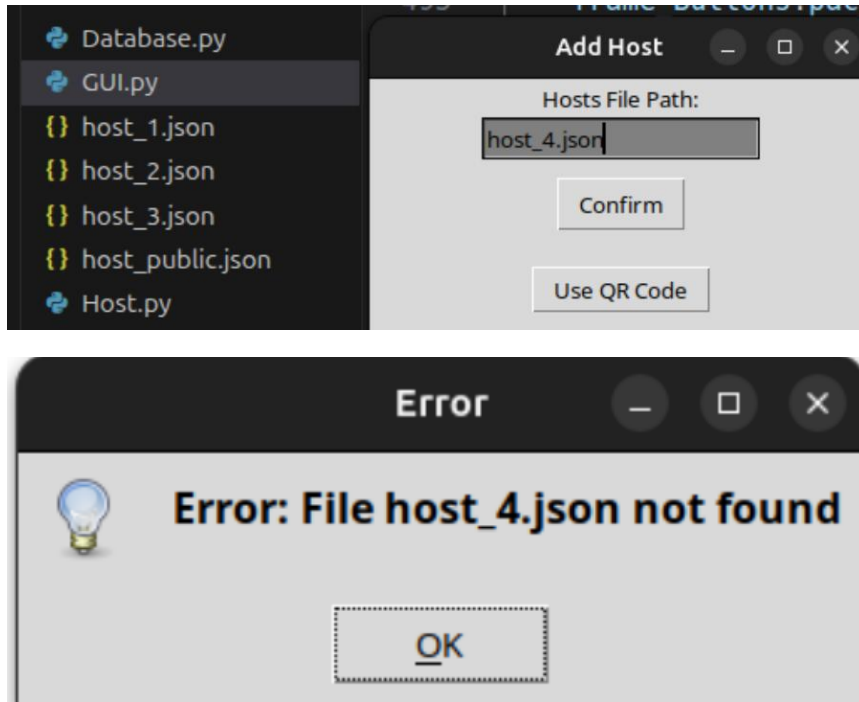
Temos também a opção de visualizar graficamente os resultados dos testes (pings) de cada host ao longo de um determinado intervalo de tempo, através da pesquisa de um host (IP) pela barra de pesquisa.

Na parte inferior da janela principal, temos vários botões que vão executar as seguintes funções: Exportar os resultados de um ou de todos os hosts; Adicionar Host (via ficheiro de configuração ou QR Code); Remoção de um Host (inserindo o seu IP); Adicionar um Admin (preenchendo os seus 5 campos); e Remoção de um Admin (inserindo o seu email).

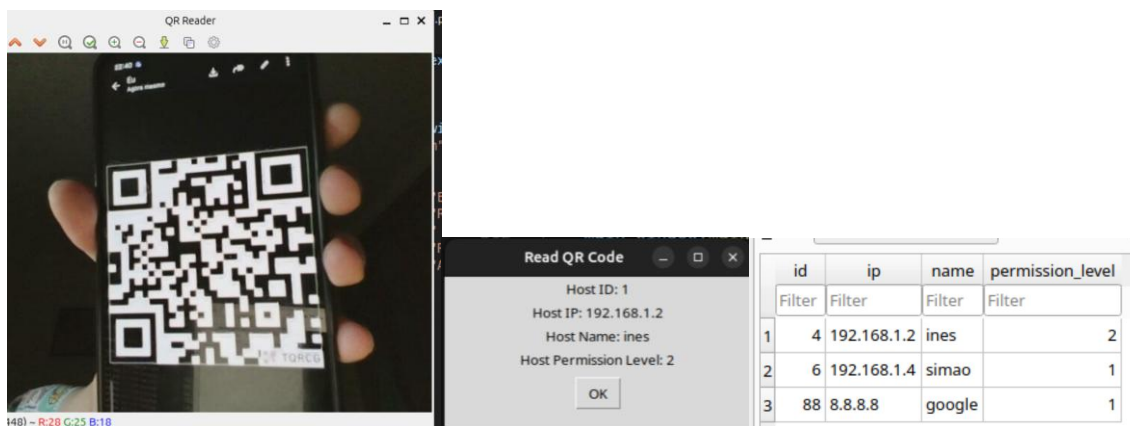


Testes e resultados

Ao se tentar adicionar um host, escrevendo um caminho para um ficheiro de configuração que não existe, aparece um pop-up com uma mensagem de erro:



Ao se tentar adicionar um host através de um QR Code, é possível verificar que a leitura é feita corretamente, no entanto, existe um erro ao guardar essa informação na tabela Hosts da base de dados (através da segunda foto é possível ver que o host com 'id' 2 não foi adicionado):



Adicionar um novo admin:

Add Admin

Name:

ines

Email:

i@gmail.com

Password:

1234

SMTP:

gmail

Permission Level:

2

Confirm

Table: Admins

	name	email	password	server	permission_level
	Filter	Filter	Filter	Filter	Filter
1	ines	i@gmail.com	gmail	2	1234

Remoção de um Admin:

Remove Admin

Email:

i@gmail.com

Confirm

Admin removed with success!

OK

Table: Admins

	name	email	password	server	permission_level
	Filter	Filter	Filter	Filter	Filter

Remover um host:

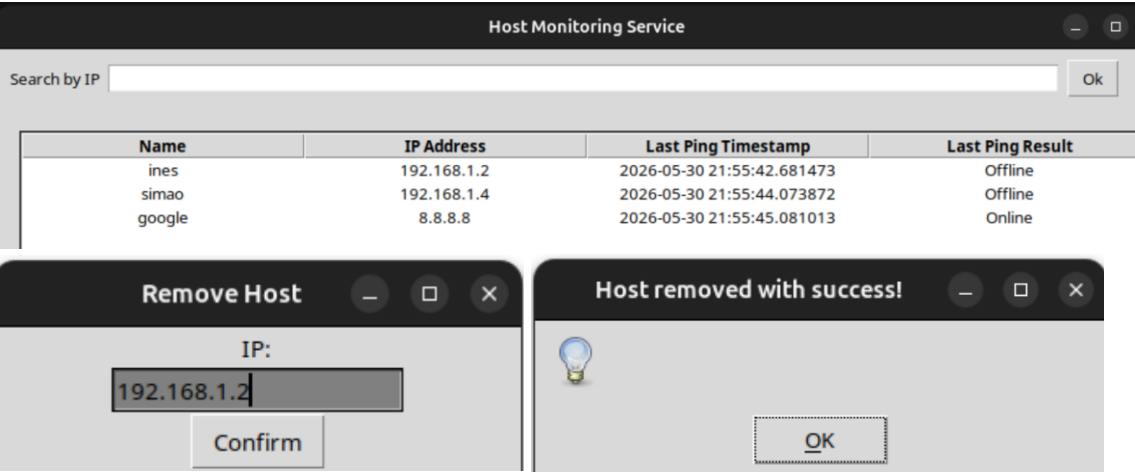
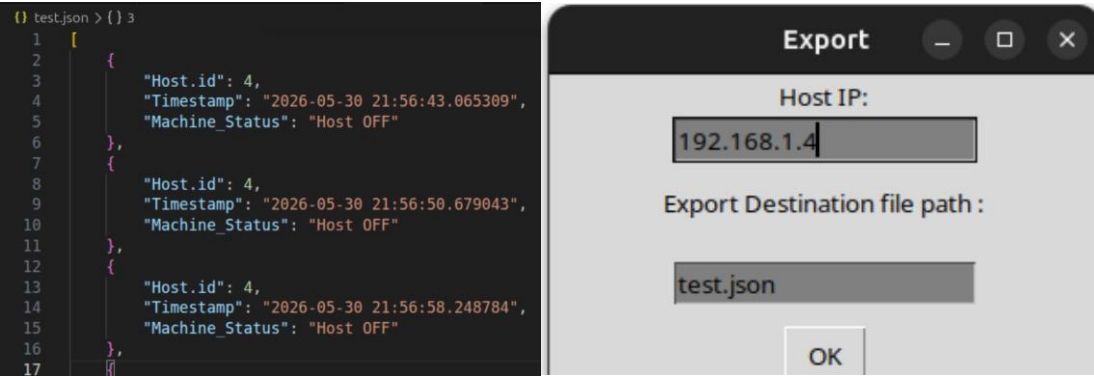


Table: Hosts

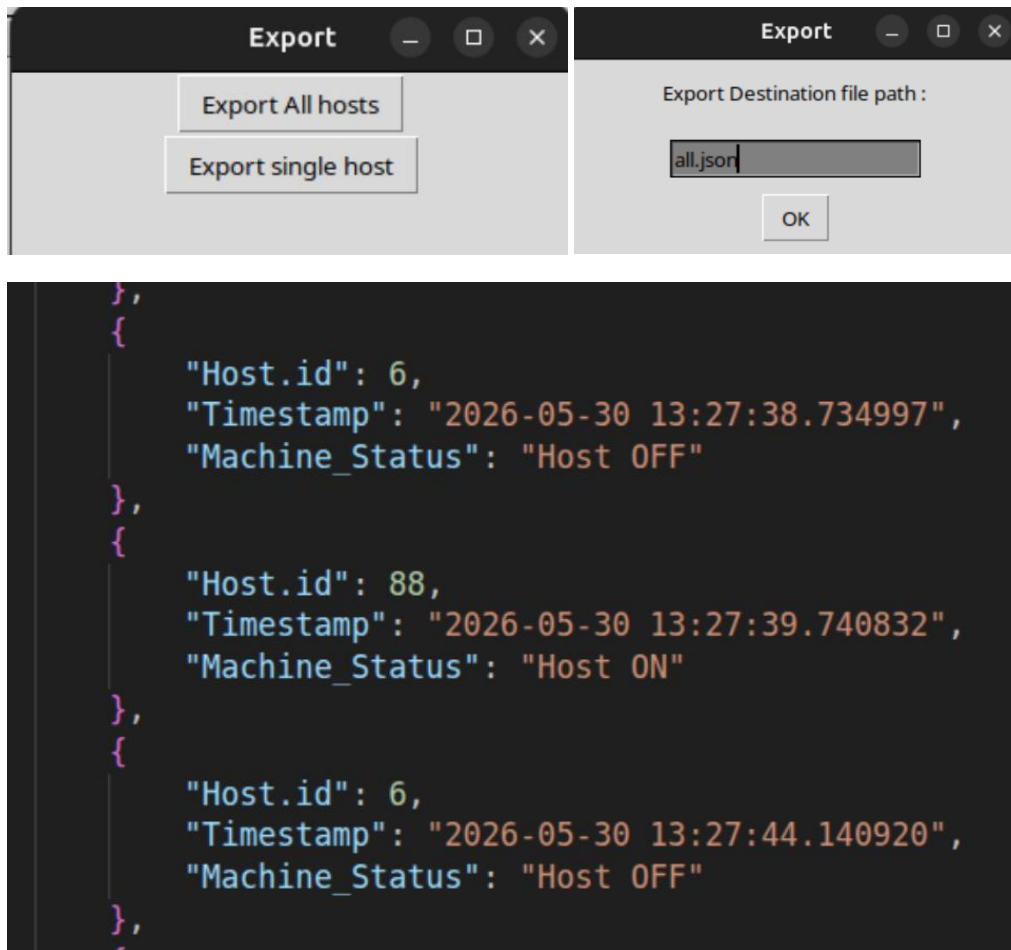
	id	ip	name	permission_level
	Filter	Filter	Filter	Filter
1	6	192.168.1.4	simao	1
2	88	8.8.8.8	google	1

No entanto, para se conseguir visualizar a alteração pela interface gráfica, é necessário fechar o programa e voltar a abri-lo.

Exportar os resultados de todos os testes de hosts existentes:

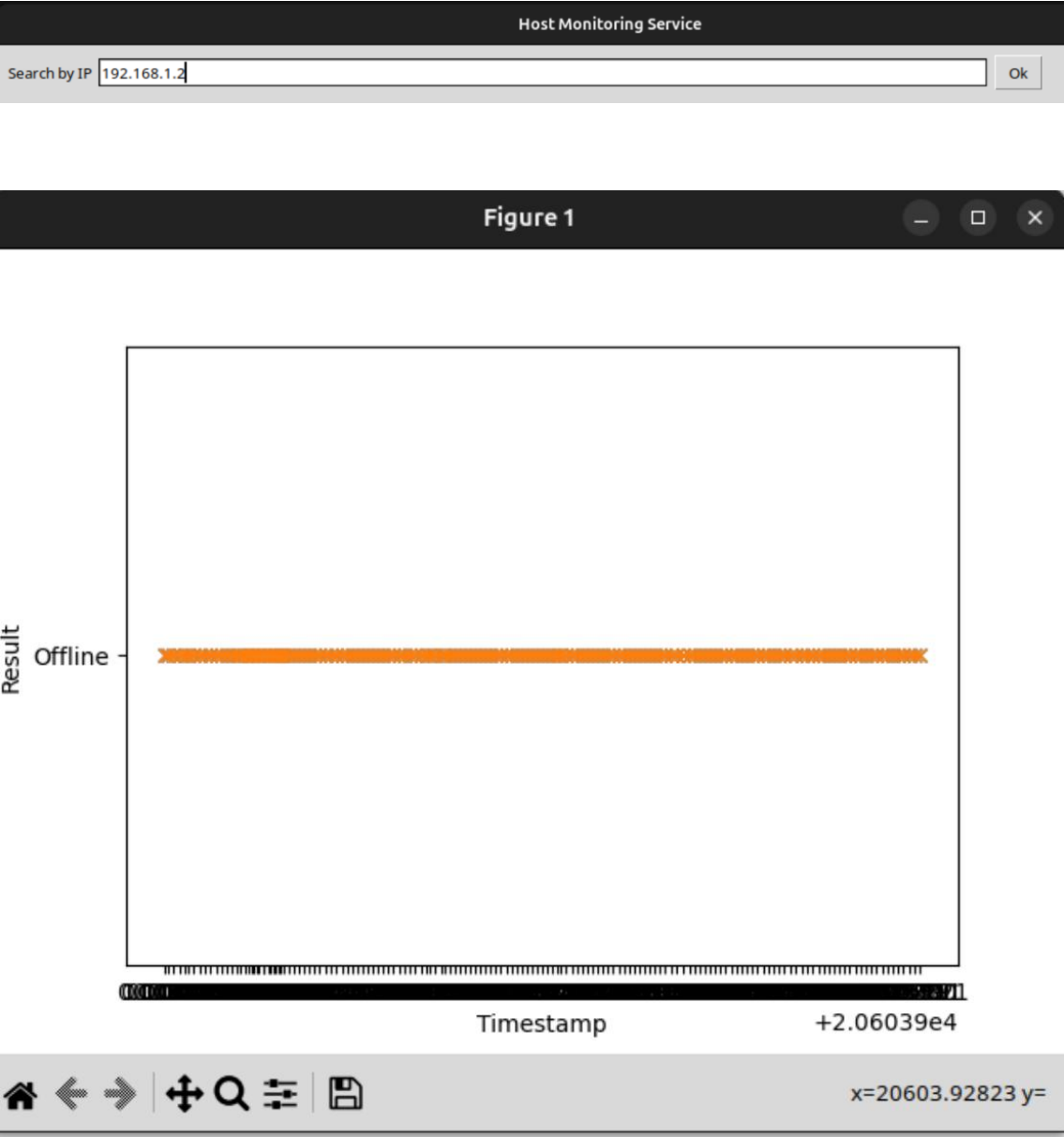


Exportar os resultados de todos os testes de hosts existentes:

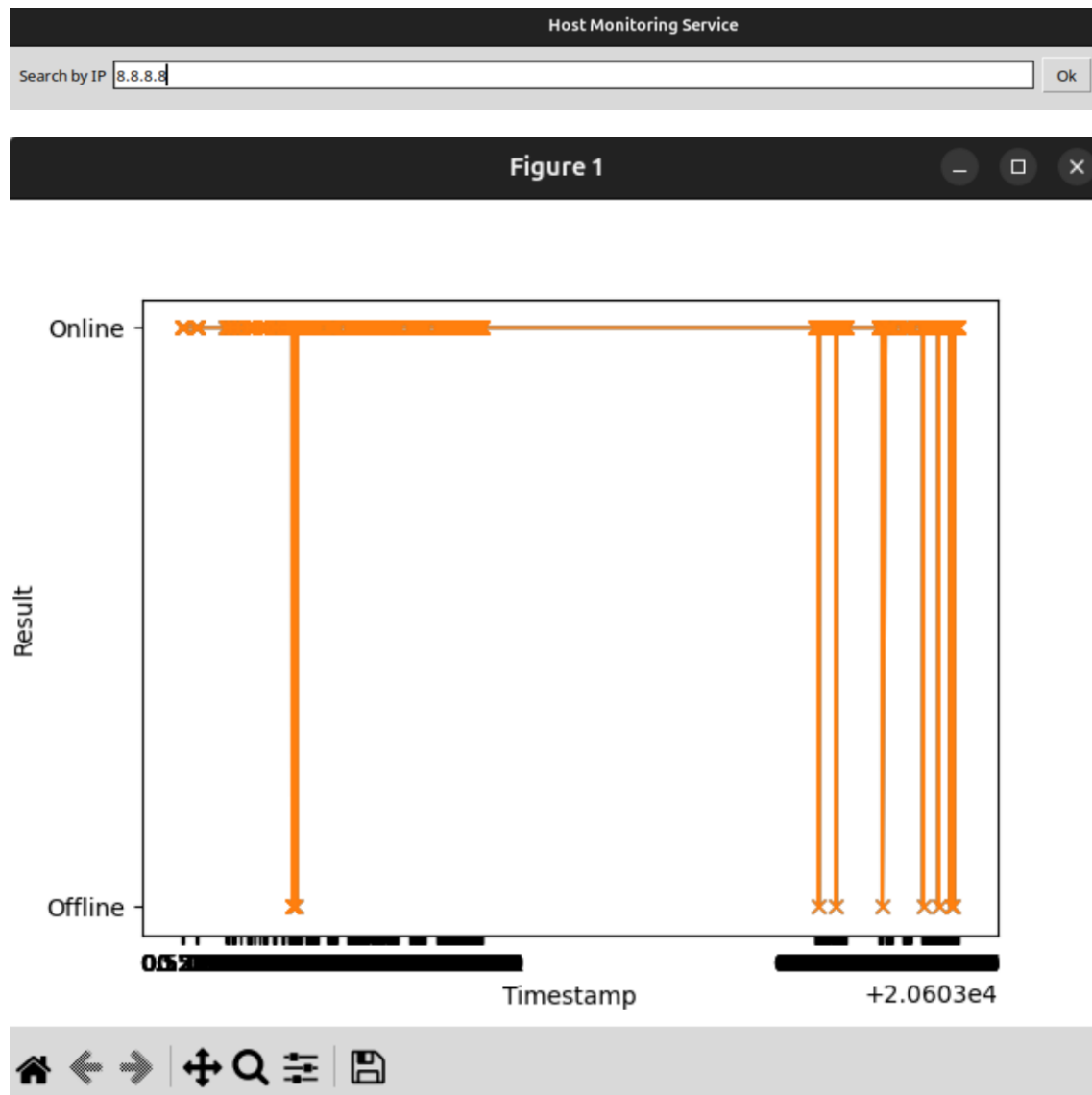


Como a execução do programa estava muito lenta, optámos por reduzir o número de pacotes enviados por cada ping, e adicionámos um 'timeout limit'. Assim, tornámos a execução mais rápida.

Verificação do gráfico com os testes ao longo do tempo de execução do programa para o host 192.168.1.2:



Verificação do gráfico com os testes ao longo do tempo de execução do programa para o host com IP 8.8.8.8:



Peer Evaluation

Ricardo (33,333%): Monitor Service, correções Database, Testes, UML, relatório

Inês (33,333%): GUI, Database, UML, Modelo Relacional, QR Code, relatório

Simão (33,333%): correções Monitor Service, GUI, Admins, Hosts, Database